

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: “OSLearn” – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).

- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.
- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.
- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

E.Student Work

1. Problem Understanding and Formulation

- **OSLearn** is an interactive web application for learning Operating System algorithms.
- It helps students understand **Process Scheduling, Memory Management, and Disk Scheduling** practically.
- Users can enter inputs and view **step-by-step results, tables, Gantt charts, and comparisons**.
- The main objective is to make OS algorithm learning **simple, visual, interactive, and easy to understand**.

2. Application of Course Knowledge (CO2, CO3, CO4)

The project applies concepts learned in Operating Systems, particularly process management, memory management, file systems, and disk scheduling.

Part I – Process Synchronization & Deadlock (CO2, Unit II)

Process management concepts are applied to understand how processes compete for CPU resources. The project uses scheduling algorithms such as **FCFS, SJF, and Round Robin** to determine the order in which processes receive CPU time.

Although the main implementation focuses on process scheduling, the application can also be extended to demonstrate synchronization and deadlock concepts such as **mutual exclusion, semaphore-based synchronization, resource allocation, and deadlock detection**. These concepts help students understand how operating systems coordinate multiple processes and avoid unsafe resource allocation.

Part II – Memory Management (CO3, Unit III)

Memory management concepts are implemented using **page replacement algorithms**. The system supports **FIFO and LRU** algorithms. Users provide the number of frames and a page reference string. The application displays the contents of memory after each page reference and identifies whether the reference causes a page fault.

This helps demonstrate concepts such as **paging, page faults, frames, page replacement, and memory utilization**.

Part III – File Systems & Disk Scheduling (CO4, Unit IV)

Disk scheduling concepts are applied through **FCFS, SSTF, and SCAN** algorithms. The user provides a disk request queue and initial disk-head position. The application calculates the order in which requests are serviced and the total seek time.

These concepts are related to file-system performance because efficient disk access directly affects file reading, writing, and storage operations.

3.Solution / Design / Methodology

The system follows:

User Input → Algorithm Selection → Processing → Result → Visualization

The application accepts process details, page reference strings, and disk requests. Python functions execute the selected algorithms and generate performance results such as waiting time, turnaround time, page faults, and seek time.

4. Use of Modern Tools

The project uses:

- **Python** – Algorithm implementation
- **Streamlit** – Web application
- **Pandas** – Data tables
- **Matplotlib** – Charts and Gantt charts
- **VS Code** – Development
- **GitHub** – Code submission and version control

5. Results and Validation

The application was tested using sample inputs for all three modules. The generated results were compared with manual calculations.

The system successfully displays:

- Gantt charts and scheduling metrics
- Page replacement steps and page faults
- Disk seek sequence and total seek time

Different test cases were used to check correctness and handle common inputs

6. Analysis and Engineering Decision

FCFS is simple but may have higher waiting time. SJF can reduce waiting time, while Round Robin provides better response for interactive systems. LRU generally performs better than FIFO for many reference patterns. SSTF reduces seek distance, while SCAN provides more balanced disk scheduling.

The algorithm should be selected based on the system requirement and workload.

7. Broader Considerations

OSLearn supports interactive learning and improves understanding through visualization. It provides transparent intermediate steps and reproducible results. The application can also be extended with more OS algorithms in the future.

8. Conclusion

OSLearn successfully provides an interactive platform for learning and comparing Operating System algorithms. It combines Python and Streamlit to demonstrate scheduling, memory management, and disk scheduling concepts in a simple and understandable way.

9. Student Reflection

This project helped me understand OS algorithms through practical implementation. I learned how scheduling, page replacement, and disk scheduling work and how their performance can be compared. Developing the Streamlit application also improved my Python programming and problem-solving skills.

10. References

Cite all textbooks, datasets, standards, software/simulation tools, and any AI-assisted tools used, in a consistent citation format (e.g., IEEE).

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10
Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

